BCSE-1501
Network Security

Cryptography & Trust Management

What is Cryptography?

Cryptography is the science of securing information by transforming readable data (**plaintext**) into an unreadable form (**ciphertext**) using **mathematical algorithms and secret keys**, ensuring that only authorized parties can access or verify the data.

Core Objectives

- **Confidentiality** – Prevents unauthorized data disclosure
- **Integrity** – Ensures data is not altered in transit
- **Authentication** – Verifies the identity of communicating parties
- **Non-repudiation** – Prevents denial of actions (e.g., digital signatures)

How It Works

- Plaintext → **Encryption Algorithm + Key** → Ciphertext
- Ciphertext → **Decryption Algorithm + Key** → Plaintext

Why Cryptography is Essential

- Secures **internet communication (HTTPS, TLS)**
- Protects **passwords, transactions, emails, cloud data**
- Forms the backbone of **trust in distributed systems**

GLA UNIVERSITY MATHURA

Trust in Distributed Systems

Trust is the assumption that a remote entity will behave as expected, even in the presence of **partial failures, malicious nodes, or untrusted networks**.

Why Trust is Challenging

- No single global authority
- Nodes operate independently
- Communication over insecure networks
- Possibility of compromised or malicious participants

Foundations of Trust

- **Authentication** – Who are you?
- **Authorization** – What are you allowed to do?
- **Integrity** – Has data been altered?
- **Availability** – Can services be reliably accessed?

How Trust is Established

- **Cryptography:** Encryption & digital signatures
- **PKI:** Certificate Authorities and trust chains
- **Protocols:** TLS, Kerberos
- **Models:** Zero Trust, Web of Trust

GLA UNIVERSITY MATHURA

Symmetric Cryptography

Symmetric Cryptography is an encryption technique in which the **same secret key** is used for both **encryption and decryption** of data.

How It Works

- Sender encrypts plaintext using a **shared secret key**
- Encrypted data (ciphertext) is transmitted
- Receiver decrypts using the **same key**

Key Characteristics

- **Fast and efficient** (suitable for large data)
- Requires **secure key distribution**
- Key compromise breaks confidentiality

Types

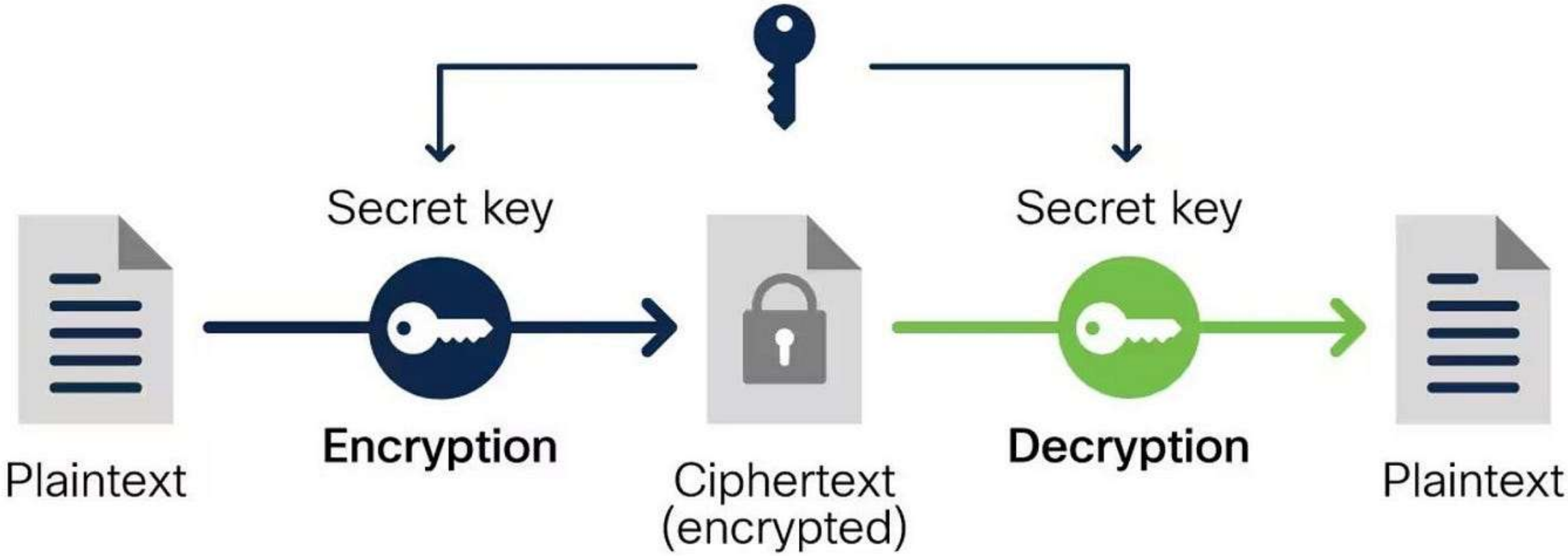
- **Block Ciphers:** AES, DES
- **Stream Ciphers:** RC4 (deprecated), ChaCha20

Weaknesses of Symmetric Cryptography (Short)

- **Key distribution problem** – Securely sharing the secret key is difficult.
- **Poor scalability** – Requires many keys as the number of users increases.
- **Single key compromise** – If the key is leaked, all encrypted data is exposed.

GLA UNIVERSITY MATHURA

Symmetric encryption



Asymmetric Cryptography

Asymmetric Cryptography uses a **pair of mathematically related keys**:

- **Public Key** – shared openly
- **Private Key** – kept secret by the owner

How It Works

- Data encrypted with a **public key** can only be decrypted using the corresponding **private key**
- Enables secure communication without prior key sharing

Key Features

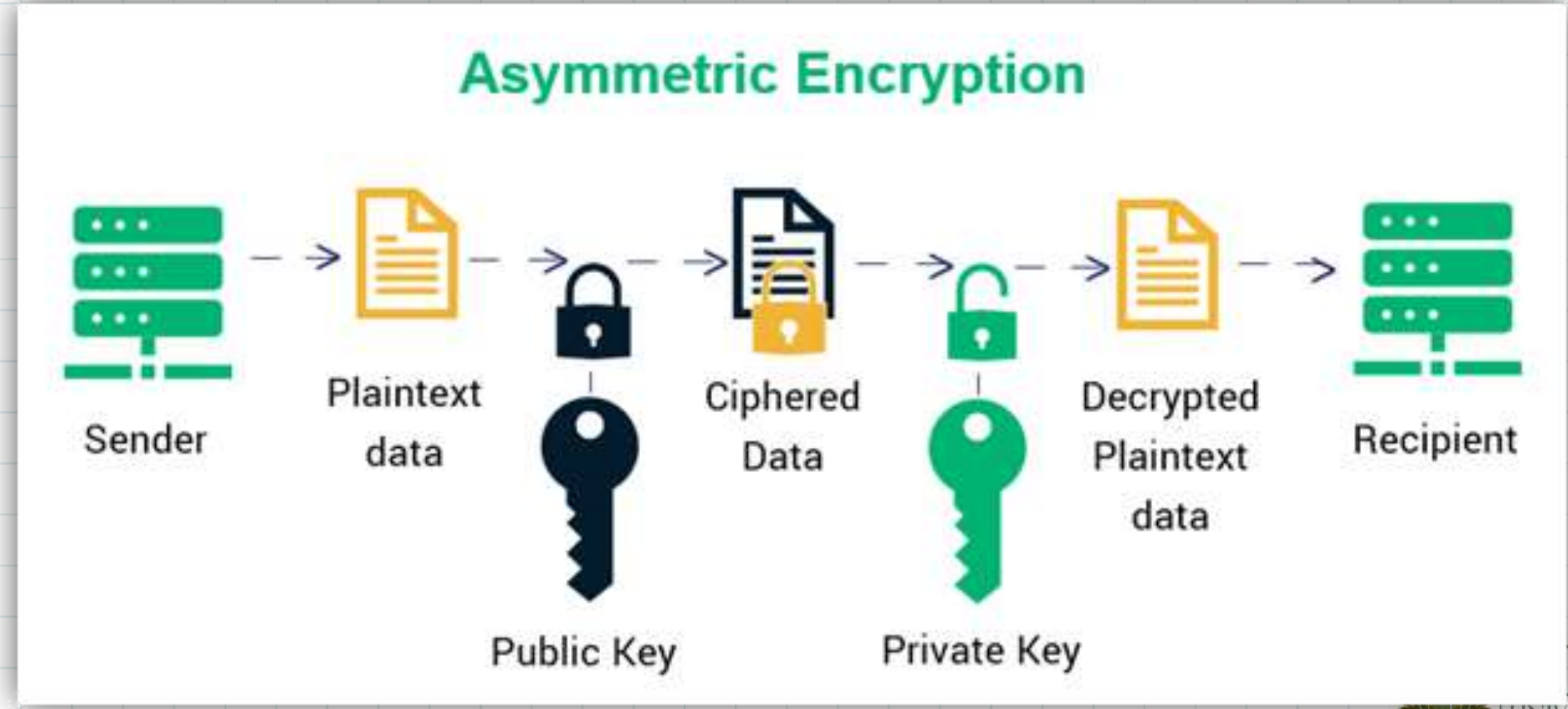
- Solves the **key distribution problem**
- Supports **authentication and digital signatures**
- Computationally **slower** than symmetric cryptography

Common Algorithms

- RSA
- Diffie–Hellman (Key Exchange)

GLA UNIVERSITY MATHURA

Asymmetric Key Cryptosystems (Public Key Cryptography)



Public Key

The public key is used to encrypt messages.

It can be shared with anyone who wants to send a secure message to the key owner.

Private Key

The private key is used to decrypt messages that were encrypted with the corresponding public key.

It must be kept secret to ensure that only the intended recipient can read the encrypted messages.

Symmetric vs Asymmetric Cryptography

Aspect	Symmetric Cryptography	Asymmetric Cryptography
Keys Used	Single shared secret key	Public–private key pair
Encryption Speed	Very fast	Slower
Key Distribution	Difficult and insecure	Easy
Scalability	Poor	Good
Authentication	Not inherent	Supported
Digital Signatures	Not supported	Supported
Computational Cost	Low	High
Common Algorithms	AES, DES	RSA, ECC
Typical Use	Bulk data encryption	Key exchange, authentication
Real-World Example	Disk encryption, VPN tunnels	HTTPS, PKI, email security

GLA UNIVERSITY MATHURA

HASH FUNCTIONS

A **Hash Function** is a mathematical function that transforms an input message of **arbitrary length** into a **fixed-length output**, known as the **hash value**, **digest**, or **message digest**.

$$H: \{0,1\}^* \rightarrow \{0,1\}^n,,$$

Where:
H = hash function,
input = any binary string of arbitrary length,
output = fixed-length string (n bits).
Hash functions are widely used in cryptography to ensure **data integrity**, **authentication**, and **digital signature verification**.

GLA UNIVERSITY MATHURA

SHA (Secure Hash Algorithm)

SHA is a family of **cryptographic hash functions** designed to ensure **data integrity and authenticity** by producing a fixed-length hash from input data of arbitrary size.

SHA Family

- **SHA-1** – 160-bit hash *(deprecated)*
- **SHA-2** – SHA-224, **SHA-256**, SHA-384, SHA-512
- **SHA-3** – Latest standard, different internal design

Key Properties

- One-way (irreversible)
- Fixed-length output
- Collision-resistant *(for SHA-2, SHA-3)*
- Deterministic

Security Status

- ❌ SHA-1: Vulnerable to collision attacks
- ✓ SHA-256 / SHA-3: Secure and widely recommended

Real-World Uses

- Password hashing (with salt)
- Digital signatures
- TLS/SSL, Blockchain
- File and message integrity verification

GLA UNIVERSITY MATHURA

MD5 (Message Digest Algorithm 5)

MD5 is a **cryptographic hash function** that produces a **128-bit (16-byte) hash value** from an input of arbitrary length.

Key Characteristics

- Fixed-length output (128-bit hash)
- Deterministic (same input → same hash)
- Fast computation
- One-way function (no decryption)

Intended Security Properties

- Pre-image resistance
- Second pre-image resistance
- Collision resistance ✗ *(broken)*

Major Weakness

- **Collision attacks** are practical
- Two different inputs can generate the **same hash**
- Not secure for cryptographic integrity or authentication

GLA UNIVERSITY MATHURA

AES (Advanced Encryption Standard)

AES is a **symmetric block cipher** standard used to securely encrypt data.

Key Features

- Block size: **128 bits**
- Key sizes: **128 / 192 / 256 bits**
- Very fast and secure
- Resistant to known practical attacks

Used In

- Disk & database encryption
- VPNs
- TLS session encryption
- Wi-Fi security (WPA2/WPA3)

Why is it so strong?

- **It's a "Round" System:** AES doesn't just do those steps once. It does them **10 to 14 times** (called "Rounds").
- **Math Power:** To "guess" the key for the strongest version (AES-256), a supercomputer would need to work for billions of years.
- **Everywhere:** It's used by your phone (iMessage/WhatsApp), your bank, and even the government for "Top Secret" files.

GLA UNIVERSITY MATHURA

RSA (Rivest–Shamir–Adleman)

RSA is an **asymmetric cryptographic algorithm** based on the mathematical difficulty of factoring large prime numbers.

Key Features

- Uses **public–private key pair**
- Enables **secure key exchange**
- Supports **digital signatures**
- Slower than symmetric encryption

Used In

- TLS handshakes
- Digital certificates
- Secure email
- Authentication systems

If Person **A** wants to send a message securely to Person **B**:

- Person A **encrypts** the message using Person B's **Public** Key.
- Person B **decrypts** the message using their **Private** Key.

Key Generation

Select two distinct prime numbers p and q

Compute $n=p\times q$, , where n is called the modulus for encryption and decryption.

Compute $\varphi(n)=(p-1)\times(q-1)$

Choose an integer e such that $1 < e < \varphi(n)$ and $GCD(e,\varphi(n))=1$ (i.e., e is co-prime to $\varphi(n)$).

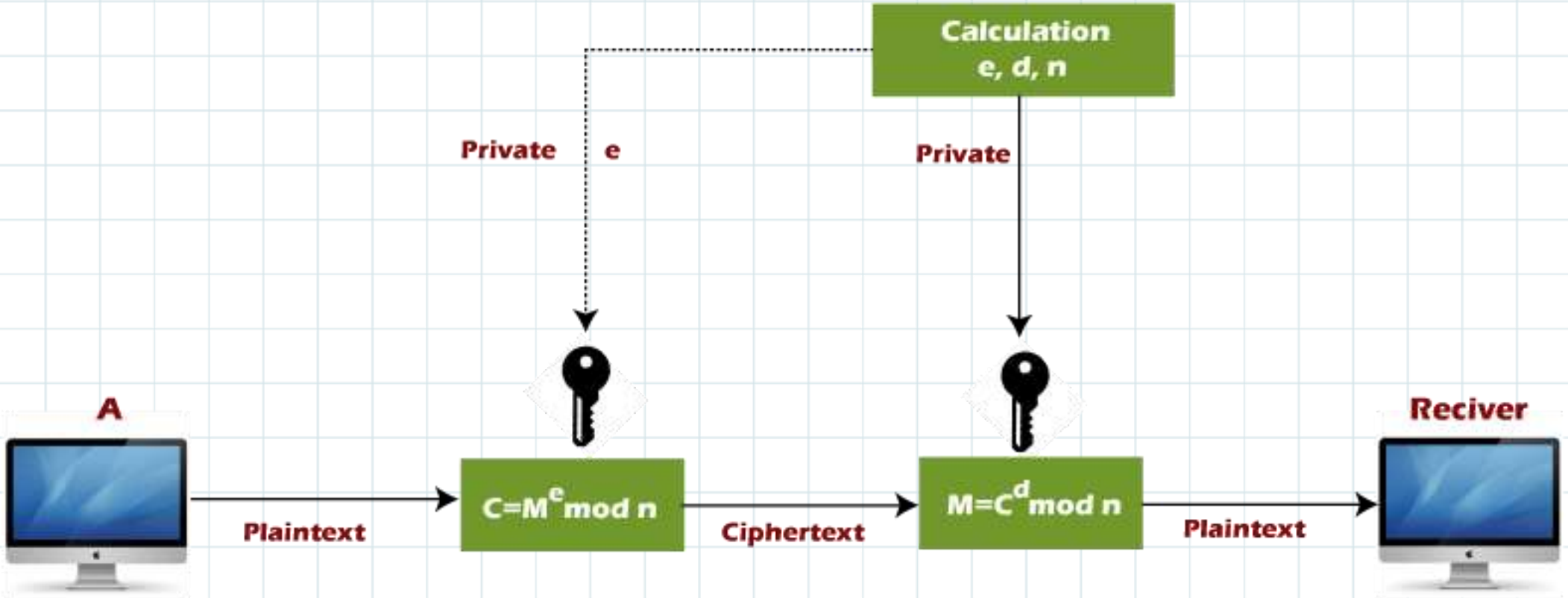
Find d: Satisfies $(e\times d) \bmod \varphi(n)=1$

Encryption:

Ciphertext $c = m^e \bmod n$, where m is the plaintext.

Decryption:

Plaintext $m = c^d \bmod n$


Calculation
e, d, n
Private
e
Private
A
Reciver
C=M^e mod n
M=C^d mod n
Plaintext
Ciphertext
Plaintext

RSA

Example

Step 1: in the first step, select two large prime numbers, **p** and **q**.

p = 3

q = 11

Step 2: we calculate

n = p x q

n = 3 x 11

n = 33

where **n** is called the modulus for encryption and decryption.

GLA UNIVERSITY MATHURA

Example

Step 3: calculate

$$\varphi(n) = (p - 1) * (q-1)$$

$$\varphi(n) = (3 - 1) * (11 - 1)$$

$$\varphi(n) = 2 * 10$$

$$\varphi(n) = 20$$

Example

Step 4: We need to choose e such that $1<e<\phi(n)$ and $GCD(e,\phi(n))=1$ (i.e., e is co-prime with $\phi(n)$).

Let's choose e=3, as it satisfies

$GCD(3,20)=1$

GLA UNIVERSITY MATHURA

Example

Step 5: Find d

$(e \times d) \bmod \phi(n) = 1$

To find d, we solve the equation $(3 \times d) \bmod 20$

By trial, **d=7** satisfies this equation, as

$(3 \times 7) \bmod 20$

$= 21 \bmod 20 = 1$

Example

Step 6: Public and Private Keys

Public Key: (e=3,n=33)
Private Key: (d=7,n=33)

Example

Step 7: Encryption

To encrypt a message m, use the formula:

$$c = m^e \bmod n$$

Let's encrypt the message m=7

$c=7^3 \bmod 33$

$c=343 \bmod 33$

$c= 343 \div 33 = 10$ remainder 13

So, c=13.

Encrypted message: c=13

GLA UNIVERSITY MATHURA

Example

Step 6: Decryption

To decrypt the ciphertext c=13, use the formula:
$m = c^d \bmod n$

$m=13^7 \bmod 3$
$=62,748,517 \bmod 33$
$= 7$ which is the original message.

GLA UNIVERSITY MATHURA

SSL (Secure Sockets Layer)

The Secure Socket Layer (SSL) is a cryptographic protocol designed to provide secure communication over a computer network. It was developed by **Netscape** in the 1990s to establish an encrypted link between the web server and a web browser. SSL operates by using encryption to secure the transmission of data ensuring that sensitive information such as credit card details and personal data remains confidential.

Key Features of SSL

- **Encryption**: It uses encryption algorithms to protect data during transmission.
- **Authentication**: It verifies the identity of the server to ensure that data is sent to the correct destination.
- **Data Integrity**: It ensures that data has not been altered during the transmission.
- **Fortezza algorithm**: The Fortezza algorithm is a cryptographic system that was used in early versions of SSL (like SSL 3.0) to support strong encryption for secure communications.

GLA UNIVERSITY MATHURA

Transport Layer Security (TLS)

The Transport Layer Security (TLS) is the successor to SSL and is designed to provide improved security and efficiency. TLS was developed as an enhancement of SSL to the address various vulnerabilities and to the incorporate modern cryptographic techniques.

Key Features

- Secure handshake using asymmetric crypto
- Fast data encryption using symmetric crypto
- Certificate-based authentication
- Supports Perfect Forward Secrecy

Versions

- TLS 1.2 – Widely used
- TLS 1.3 – Faster and more secure

Used In

- HTTPS
- Secure email
- VPNs
- Cloud & API security

GLA UNIVERSITY MATHURA

Public Key Infrastructure

Public Key Infrastructure (PKI) is a framework of **policies, roles, hardware, software, and procedures** used to **create, manage, distribute, validate, and revoke digital certificates**.

Core Components

- **Certificate Authority (CA)** – Issues and signs certificates
- **Registration Authority (RA)** – Verifies identity before issuance
- **Digital Certificates** – Bind public keys to identities
- **Trust Chain** – Root CA → Intermediate CA → End Entity
- **Revocation Mechanisms** – CRL, OCSP

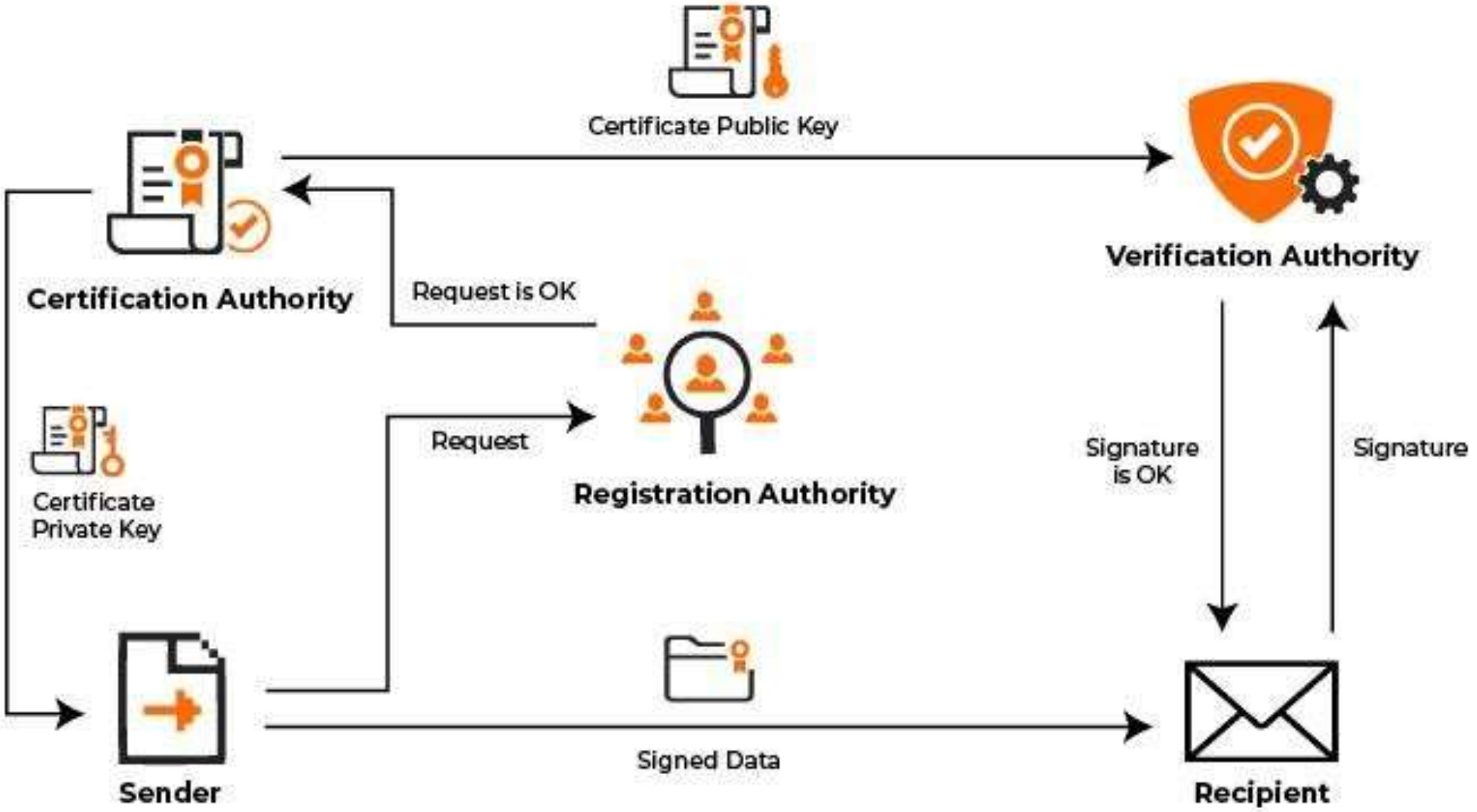
How PKI Establishes Trust

- Identity verification → Certificate issuance
- Certificate validation via trust chain
- Secure communication using public keys

Why PKI is Needed

- Enables **authentication**, **confidentiality**, and **non-repudiation**
- Prevents **impersonation** and **MITM attacks**
- Forms the trust backbone of the Internet

Public Key Infrastructure



Digital Signatures,

A **Digital Signature** is a cryptographic mechanism that uses **asymmetric cryptography and hash functions** to verify the **authenticity, integrity, and non-repudiation** of digital data.

How It Works

- Message is hashed (e.g., SHA-256)
- Hash is encrypted using the **sender's private key**
- Receiver decrypts using **sender's public key**
- Hashes are compared for verification

Security Guarantees

- **Authentication** – Confirms sender identity
- **Integrity** – Detects data modification
- **Non-repudiation** – Sender cannot deny signing

Common Algorithms

- RSA

Real-World Applications

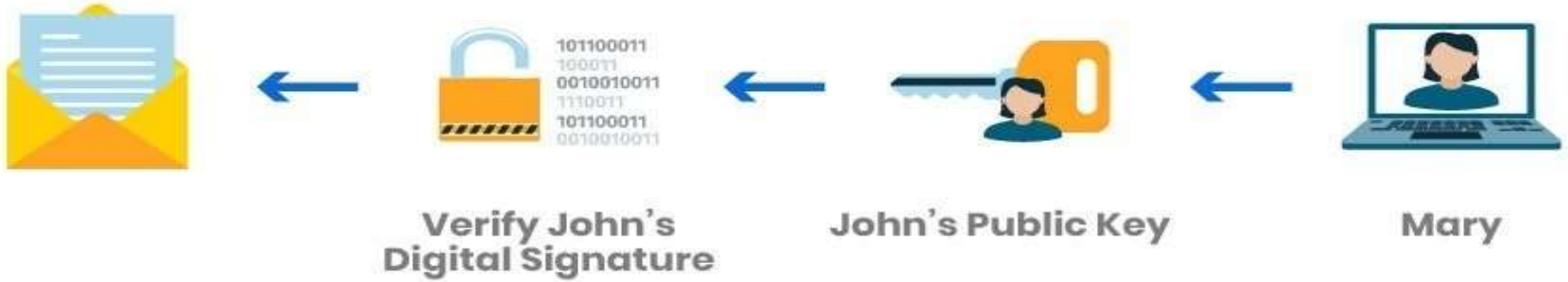
- Secure email
- Digital contracts
- Blockchain transactions

Digital Signature

1. John uses his private key to generate the digital signature of the email and sends the email to Mary.



2. Upon receiving the email, Mary verifies John's digital signature with John's public key



GLA UNIVERSITY MATHURA

Certificate Authorities (CAs)

A **Certificate Authority (CA)** is a **trusted organization** that **issues, signs, and manages digital certificates**, binding a **public key** to a verified identity.

Role of a CA

- Verifies the identity of entities (users, servers, organizations)
- Issues **X.509 digital certificates**
- Digitally signs certificates to establish trust
- Manages certificate **revocation** and **expiry**

Trust Hierarchy

- **Root CA** – Ultimate trust anchor
- **Intermediate CA** – Delegated authority
- **End-Entity Certificate** – Server/User certificate

(Trust flows from Root → Intermediate → End Entity)

Why CAs are Important

- Prevent **impersonation** and **MITM attacks**
- Enable secure **TLS/HTTPS communication**
- Form the backbone of **Internet trust**

Certificate Revocation

GLA UNIVERSITY MATHURA

Cryptographic Key Management

Cryptographic Key Management refers to the **secure generation, storage, distribution, rotation, and revocation of cryptographic keys** throughout their lifecycle.

Key Lifecycle

- **Key Generation** – Strong randomness, secure algorithms
- **Key Distribution** – Secure sharing of keys (e.g., RSA, Diffie–Hellman)
- **Key Storage** – Protected using HSM, TPM, secure vaults
- **Key Rotation & Expiry** – Limits damage from key compromise
- **Key Revocation** – Invalidates compromised or expired keys

Key Distribution Methods

- Manual / Pre-shared keys (small systems)
- **Asymmetric key exchange** (TLS, PKI)
- Centralized systems (Kerberos, KMS)

Why It Is Critical

- Weak key management breaks **strong cryptography**
- Prevents **key leakage**, **impersonation**, and **MITM attacks**
- Essential for **cloud, enterprise, and IoT security**

GLA UNIVERSITY MATHURA

Cryptographic Key Distribution

Cryptographic Key Distribution is the process of **securely sharing cryptographic keys** between communicating parties so that encrypted communication can take place.

Why It Is Challenging

- Communication occurs over **untrusted networks**
- Keys must remain **confidential and authentic**
- Vulnerable to **Man-in-the-Middle (MITM) attacks**

Common Key Distribution Methods

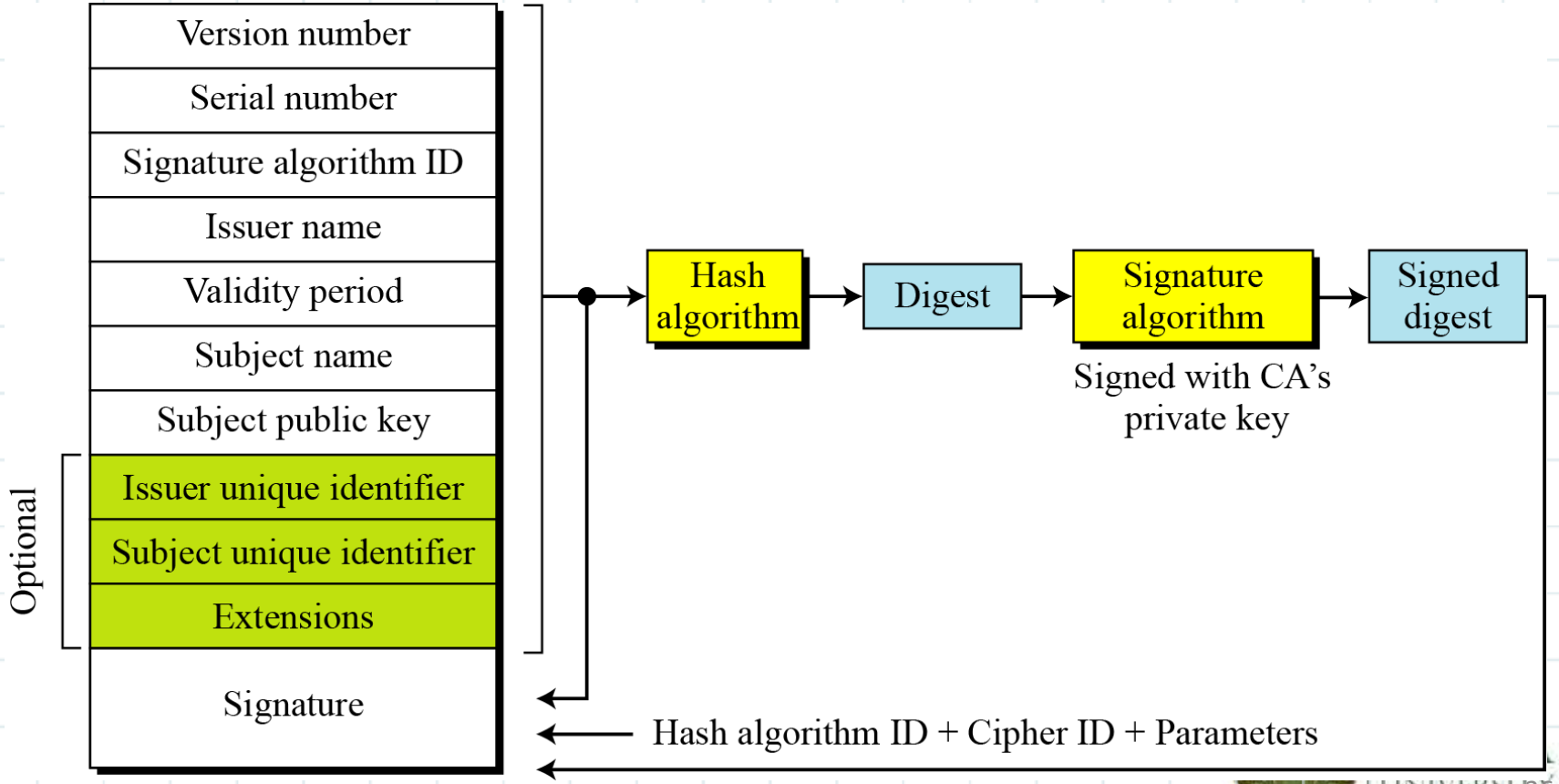
- **Pre-shared Keys (PSK)** – Simple but poor scalability
- **Asymmetric Key Exchange** – RSA, Diffie–Hellman
- **Trusted Third Party** – Kerberos, centralized servers
- **PKI-based Distribution** – Certificates and CAs

Real-World Examples

- TLS uses asymmetric cryptography to exchange symmetric keys
- Kerberos distributes session keys via a trusted server
- Wi-Fi security uses PSK or enterprise authentication

X.509 Certificates

An **X.509 certificate** is a digital certificate that uses the **X.509 public key infrastructure (PKI) standard** to verify the identity of a party and securely associate that identity with a **public key**. It's most commonly used in **SSL/TLS**, **HTTPS**, **email security (S/MIME)**, **code signing**, and **VPNs**.

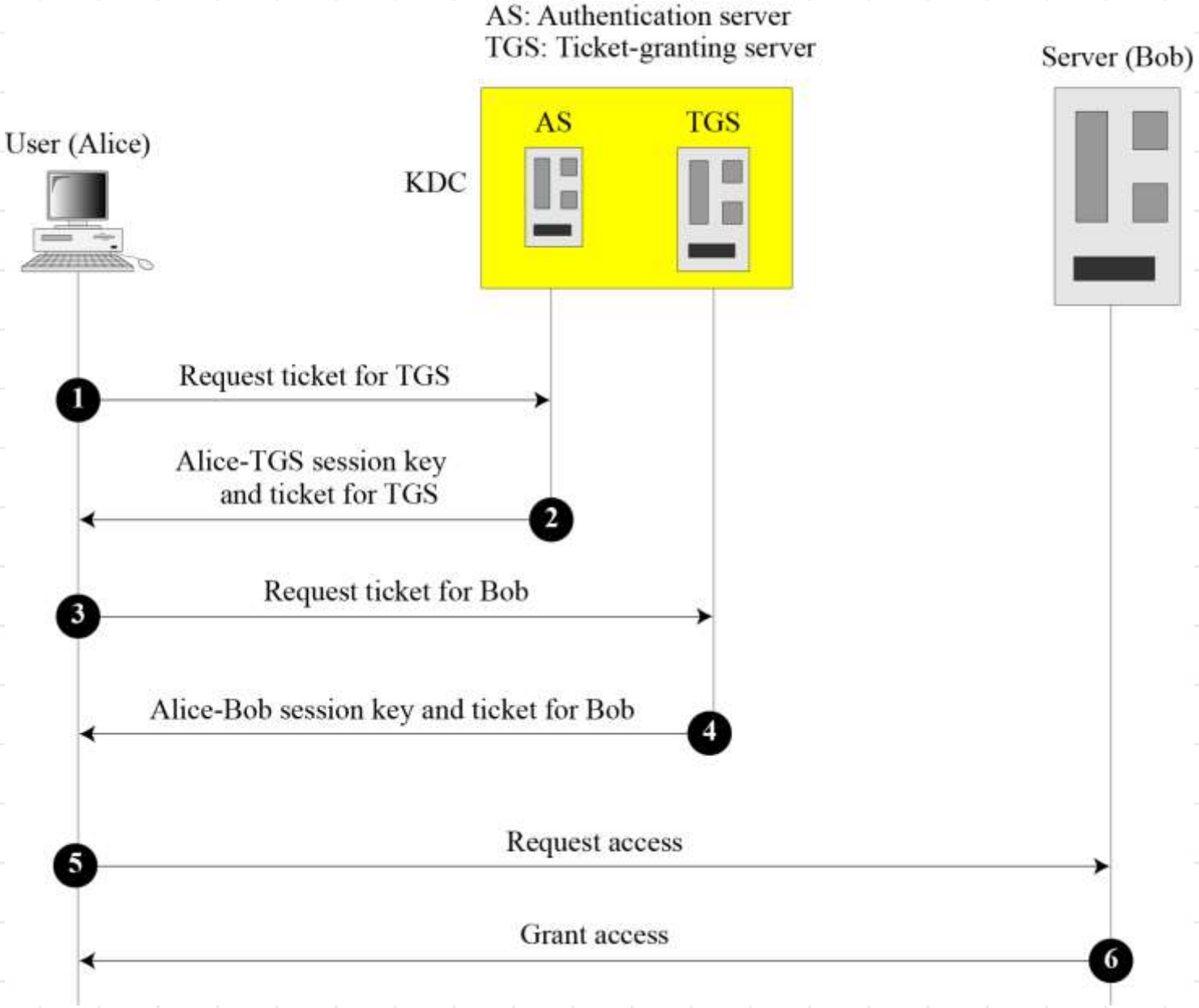


Kerberos

Kerberos is a **network authentication protocol** used in cryptography to securely authenticate users and services in an **insecure network** (like the internet or a corporate LAN). **Kerberos** provides a centralized authentication server whose function is to authenticate users to servers and servers to users. In Kerberos Authentication server and database is used for client authentication. Kerberos runs as a third-party trusted server known as the Key Distribution Center (KDC). Each user and service on the network is a principal.
It was developed by MIT and is widely used in **enterprise environments**, especially with **Microsoft Active Directory**.

- **Trusted Third Party:** The Key Distribution Center (KDC) is the heart of Kerberos.
- **Symmetric Key Cryptography:** It uses shared secret keys, not public-private key pairs like in PKI.
- **Tickets, Not Passwords:** After you authenticate, you get a "ticket" you can reuse to access multiple services without re-entering your password.

Kerberos servers



Authentication Server (AS)

The authentication server (AS) is the KDC in the Kerberos protocol.

Ticket-Granting Server (TGS)

The ticket-granting server (TGS) issues a ticket for the real server (Bob).

Real Server

The real server (Bob) provides services for the user (Alice).

GLA UNIVERSITY MATHURA